

Persistência de Dados e Refatoração

(OAT3)

Autor: Ian Dias Duque
Olavo Barros Meira Filho

Agenda

O objetivo dessa apresentação é esclarecer a refatoração do projeto

1. Item 1:
Contexto

2. Item 2:
JDBC

3. Item 3:
@Value

4. Item 4:
RowMapper

5. Item 5:
Contexto

6. Item 6:
Exceptions

7. Item 7:
Crud Repository e Generics

8. Item 8:
Diagrama

- Nesta terceira fase, abandonamos o armazenamento em memória e implementamos uma arquitetura de dados profissional.
- Migração para **MySQL**.
- Integração com **HikariCP** (Connection Pool).
- Padronização com **CrudRepository**.
- Eliminação de "Boilerplate" com **Lombok**.

- **HikariCP:** Gerenciamento de conexões de alta performance para o MySQL.
- **Lombok:** Uso de `@Data`, `@Builder` e `@AllArgsConstructor` para classes limpas.
- **JDBC Puro:** Manipulação direta de SQL sem o uso de ORMs (Hibernate/JPA).

Persistência: Repositórios JDBC

- Implementamos a lógica de banco usando PreparedStatement para garantir segurança contra SQL Injection.
- Conexões obtidas via ConnectionPool.
- Execução de queries nativas para a User Story de **Reativar Usuário**.

```
1 private Usuario update(Usuario usuario) {
2     validarEmailUnico(usuario.getEmail(), usuario.getId());
3
4     String sql = ""
5         UPDATE usuario
6         SET nome = ?, data_nascimento = ?, sexo = ?, email = ?,
7         senha = ?, tipo_usuario = ?, ativo = ?
8         WHERE id = ?
9     "";
10
11     try (Connection conn = connectionPool.getConnection();
12         PreparedStatement ps = conn.prepareStatement(sql)) {
13
14         preencherStatement(ps, usuario);
15         ps.setLong(8, usuario.getId());
16         ps.executeUpdate();
17         return usuario;
18
19     } catch (SQLException e) {
20         throw new DatabaseOperationException(
21             "Erro ao atualizar usuário com id: " + usuario.getId(), e);
22     }
23 }
```

Configurações Dinâmicas



- Mapeamos o application.properties para a classe ConfigProperties usando a annotation @Value

```
1 @Value(key = "datasource.url")  
2     private String url;  
3  
4     @Value(key = "datasource.username")  
5     private String username;
```

RowMapper: Mapeamento de Linhas



- **Tradução de Dados:** O RowMapper encapsula a complexidade de converter um ResultSet do JDBC em um objeto de domínio da aplicação.
- Isso permite que os **Services** trabalhem apenas com Objetos, ignorando a estrutura das tabelas SQL.

```
1 private Usuario mapRow(ResultSet rs) throws SQLException {
2     Usuario u = new Usuario();
3     u.setId(rs.getLong("id"));
4     u.setNome(rs.getString("nome"));
5     u.setSexo(rs.getString("sexo"));
6     u.setEmail(rs.getString("email"));
7     u.setSenha(rs.getString("senha"));
8     u.setTipoUsuario(rs.getString("tipo_usuario"));
9     u.setAtivo(rs.getBoolean("ativo"));
10
11     Date dataNasc = rs.getDate("data_nascimento");
12     if (dataNasc != null) {
13         u.setDataNascimento(dataNasc.toLocalDate());
14     }
15     return u;
16 }
```

Tratamento de Exceções Customizadas



Criamos uma hierarquia de exceções para tratar erros de negócio de forma clara e padronizada.

- Herança de RuntimeException (Unchecked).
- Captura de erros de banco de dados e conversão para mensagens de negócio.



```
1 package br.com.softhouse.dende.exceptions;
2
3 public class EmailJaCadastradoException extends RuntimeException {
4     public EmailJaCadastradoException(String email, String tipo) {
5         super(tipo + " com e-mail " + email + " já está cadastrado.");
6     }
7
8     public EmailJaCadastradoException(String message) {
9         super(message);
10    }
11 }
```


Crud Repository e Generics

- **CrudRepository:** Cada entidade (Usuário, Evento, Empresa) ganhou seu próprio repositório implementando esta interface, garantindo operações consistentes de save, findById e delete.
- Utilizamos os tipos genéricos <T, ID> para garantir que todos os nossos repositórios (Usuário, Evento, Empresa) sigam o mesmo contrato.

```

1 package br.com.softhouse.dende.repositories.util;
2
3 import java.util.List;
4 import java.util.Optional;
5
6 public interface CrudRepository<T, ID> {
7
8     T save(T entity);
9
10    Optional<T> findById(ID id);
11
12    List<T> findAll();
13
14    void deleteById(ID id);
15
16    boolean existsById(ID id);
17
18    List<T> findAllAtivos();
19 }
20

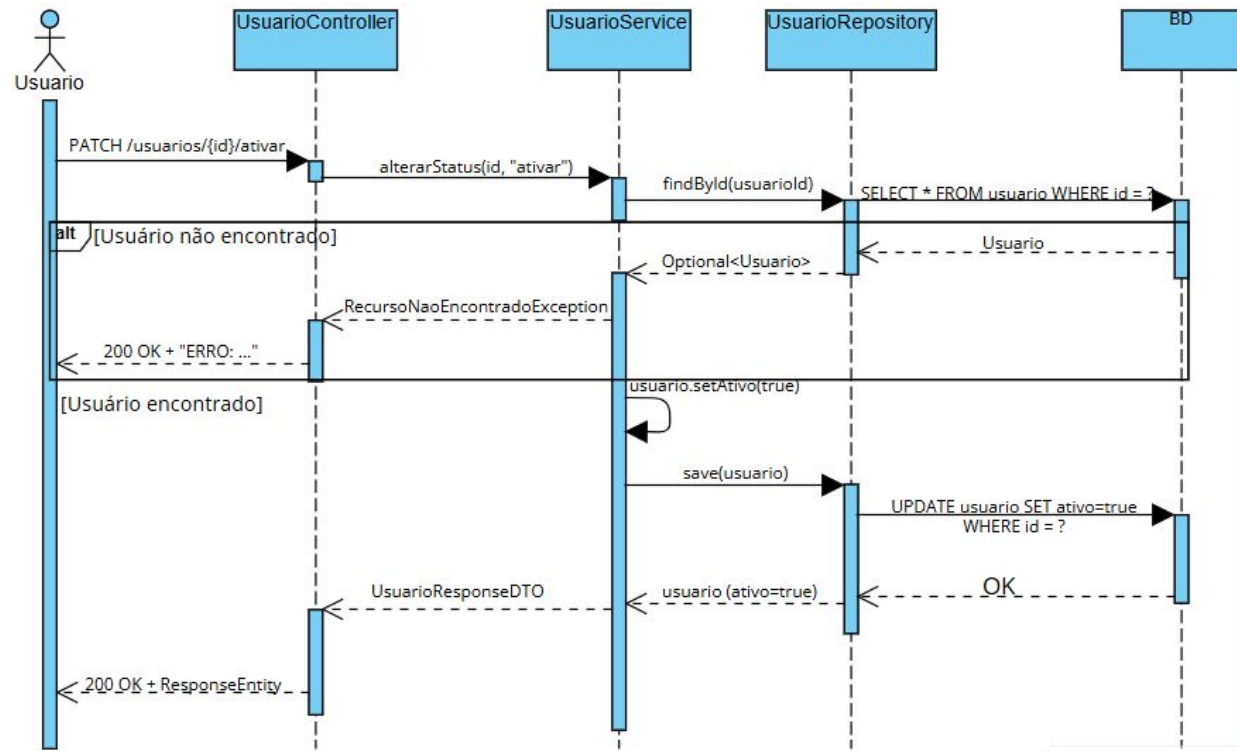
```

Diagrama de Sequência

Fluxo: Reativar Usuário

O diagrama detalha a interação entre:

- UsuarioController
- UsuarioService
- UsuarioRepository
- Banco de Dados (MySQL)



- Código 100% modularizado e livre de boilerplate.
- Persistência robusta com Pool de Conexões.
- Tratamento de exceções centralizado e eficaz.
- Pronto para escalabilidade e produção.